

- i. Declaration Page-the student should confirm the authenticity/ originality of the report and own it as his/hers, with indication that due acknowledgement was accordingly done in form of citations, quotations and references to other people's ideas/work where used.
- ii. Acknowledgement page - the student should acknowledge those who contributed the successful completion of his/her field attachment/ internship
- iii. Abstract - This provides the key issues of the students work and experiences, lessons learnt challenges, conclusions and recommendations. This should be brief and to point, but with the ability to highlight to the reader the picture of the main report.
- iv. Table of contents - This should show the contents of the report by chapters, sections and subsections with the page numbers on which such content is found in the report
- v. List of Figures - Every figure in the report should have its caption at its bottom, and such captions should be listed here with the page numbers on which the respective figures are found/ indicated. Such figures shall be numbered indicating their titles, and the numbering should follow the chapter numbers e.g., Figure 2.1 meaning figure 1 in Chapter 2.
- vi. List of Tables - All tables should be numbered and headed just like figures above, and listed on the tables' page reflecting the respective page numbers.
- vii. List of acronyms/ Abbreviations - Abbreviations and acronyms used in the report should be listed and their full meaning reflected/ given.

1.0. Chapter One: Introduction

1.1. Introduction

The field report is written document by KARAMAGI REAGAN of registration number 11/U/16268/EVE as student of MAKERERE UNIVERSITY. The student conducted the field Attachment known as Internship at Neptune Software Group for the period from 16th June to 8th August 2025.

The report comprises the coverage concepts under taken by the above-mentioned student at Neptune Software Group mainly the activities in learning the Java backend programming, embracing new software tools and frameworks.

1.2. Background of the Field Attachment

The field Attachment is field based practical work carried out for purposes of learning and researching outside the University control. Exposure to various activities, duties and responsibilities in field work organization to enhance the study programs to experience the real life of work.

1.3. Objectives of the Field Attachment

- To get hands on experience real life situation.
 - To provide an opportunity to apply the principles and techniques theoretically learnt into real-life problem-solving situations.
 - To provide an opportunity to interact with various stakeholders and potential employers.
 - To appreciate field situations that generate information for curricula review and improvement.
 - To develop understanding of work ethics, employment demands, responsibilities and opportunities.
 - To enhance and strengthen linkages between the University and various stakeholders.
-

1.4. Background of the Organisation of Field Attachment

Neptune, founded in 1999 in London, has a rich history of delivering core banking systems to the banking and financial services sector. Having delivered end to end solutions to over 100 financial service institutions over the years, have the proven experience needed to empower our clients with high performance and scalable solutions that are deployed quickly and reliably. Have been working in this industry for the last 25 years.

Neptune focus is to harness in depth business domain knowledge and technical expertise to provide innovative end to end financial services solutions. Aim to entrust financial institutions with a sustainable competitive advantage by making them more resilient and effective. Thereby, be one step ahead of their competitors.

Neptune now operates local implementation cum marketing offices across East Africa (Kenya-Regional Head office, Tanzania and Uganda) and also an office in west Africa (Nigeria). With work experience, empower clients with quick yet reliable high performance and scalable solutions.

Neptune established its Indian operation at Chennai in 2006. At Chennai Centre, take care of product engineering global product support and R&D towards, the Neptune's flagship products of Rubikon & Orbit-R.

Vision	Mission	Goals and Objectives	Values
• Empowering clients with sustainable competitive advantage by providing innovative business software solutions and excellent services within a cost active bottom line.	• Delivering client success through innovation. • Harness our deep business domain knowledge and technical expertise to provide end to end integration of technology with business processes. • Bring trust and personal responsibility to all relationships.	• To deliver core banking systems to the financial services sector. • To harness in depth domain knowledge and technical expertise. • To provide innovative end to end financial services solution. • To entrust financial institutions with a sustainable competitive advantage	• Passion • Innovation • Excellence

1.5. Organisational culture

Organizational culture refers to the shared values, beliefs, norms, behaviours, and practices that characterize a particular workplace or company. This culture is the collective personality of an organization that shapes the way employees interact, make decisions, and approach their work. Essentially, corporate culture serves as a blueprint for how business is done within the organization and influences the overall work environment and employee experiences.

Culture reflects the company's history, leadership style, and values. These cultural elements create a sense of identity and belonging among employees, as they collectively align their actions with the established norms and expectations. Office culture can be explicit, with clearly defined and communicated values. Alternatively, this culture can be implicit, existing in the unspoken behaviors and attitudes of employees. Either way, this system greatly influences employee perceptions of roles, camaraderie, collaboration, problem-solving approaches, and the organization's external reputation.

The organization's values are: Passion, Innovation and Excellence.

Neptune has a commendable work culture mainly focused on a growth mindset. The company is popular for employees dedicating their skills, time, and money to help make a difference in the world. This difference contributes to employee diversity and inclusion, considering Neptune has employees from all backgrounds and walks of life working to ensure customer satisfaction. The company's work culture deeply aligns with its core values, which include of delivering core banking systems to the banking and financial services sector.

1.6. The structure of the organisation

1.7. The main activities of the organization and ongoing IT projects

The main activities are financial software solutions automate business processes for banks and fintech companies by providing with a secure, regulatory compliant and scalable platforms to run their business efficiently.

2.2.2. Chapter Two: Student’s Experiences

2.1. Title or Position occupied in an organisation

- Security.
- Accounts.
- Sales.
- Catering.
- Marketing.
- Head technician.
- Technician.

2.2. Duties and responsibilities

Duties and responsibilities are often used interchangeably, but they have distinct meanings. Duties refer to the specific tasks or actions that an individual is expected to perform as part of their job or role. These tasks are usually outlined in a job description and are directly related to the position held. On the other hand, responsibilities encompass a broader scope and refer to the obligations and expectations that come with a particular role. Responsibilities may include duties, but they also include the overall accountability, decision-making, and management of resources associated with the role. While duties are more task-oriented, responsibilities are more encompassing and reflect the overall expectations placed on an individual.

Duties refer to the specific tasks and actions that an individual is expected to perform as part of their job or role. These tasks are often outlined in job descriptions or work contracts and are directly related to the position held. Duties are typically more concrete and tangible, focusing on the execution of specific actions or activities. They are often assigned to individuals based on their skills, expertise, and qualifications.

Responsibilities, on the other hand, encompass a broader scope and refer to the obligations and expectations that individuals have towards their roles, colleagues, and the organization as a whole. Unlike duties, responsibilities are not always explicitly defined and can vary based on the context and organizational culture. They require individuals to be accountable for their actions and the impact they have on others and the organization. Responsibilities are often more abstract and require individuals to exercise judgment and discretion in their approach.

Duties	Responsibilities
Writing code, debugging programs, and collaborating with the team to develop software solutions.	Decision making, problem-solving, and taking ownership of outcomes.

2.3. Supervision levels and relationship with Supervisor

Level of supervision is determined by the way in which work is assigned, when it is reviewed, how it is reviewed, and what guidelines, prototypes and protocols are as below.

- Close Supervision — indicates that the incumbent is assigned duties according to specific procedures. Work is checked frequently, and in addition there may be formal training.
- Supervision — indicates that the incumbent performs a variety of routine duties within established policies and procedures or by referral to the supervisor's guidelines.
- General Supervision — indicates that the incumbent develops procedures for performance of variety of duties; or performs complex duties within established policy guidelines.
- Direction — indicates that the incumbent establishes procedures for attaining specific goals and objectives in a broad area of work. Only the final results of work done are typically reviewed. Incumbent typically develops procedures within the limits of established policy guidelines.
- General Direction — indicates that the incumbent receives guidance in terms of broad goals and overall objectives and is responsible for establishing the methods to attain them. Generally, the incumbent is in charge of an area of work, and typically formulates policy for this area but does not necessarily have final authority for approving policy.

2.4. Work Team and its composition - (by positions and not individual names)

2.5. Working Relationship among team members/ other staffs

Cultivating positive workplace relationships is paramount to guarantee the prosperity of individuals and organizations. These relationships are pivotal in determining employees' job satisfaction and career advancement prospects. By acquiring a more profound comprehension of these relationships, individuals can revolutionize their professional endeavours and substantially contribute to their organization's triumph. Managing relationships with colleagues in the workplace is crucial for individuals and organizations. To achieve this, organizations must analyse the diverse types of workplace relationships.

- **Colleague Relationships:** Effective collaboration and support among colleagues are essential for a positive workplace culture. Mutual respect, trust, and effective communication form the foundation of these healthy work relationships. Supportive colleague relationships result in better teamwork, harmony, and cooperation in the organization.
 - **Supervisor-Subordinate Relationships:** For a workplace to thrive and for employees to progress in their professional trajectories, the interaction between individuals in positions of authority and their subordinates is paramount. Building a healthy work relationship with your supervisor helps you understand their expectations, receive feedback, and gain support in achieving your professional goals.
 - **Cross-Functional Relationships:** Collaboration between different teams is crucial for achieving success in projects. Cross-functional relationships involve individuals from different departments working collectively toward shared objectives. Such healthy relationships foster the exchange of ideas and expertise throughout the organization, resulting in enhanced project outcomes and the emergence of innovative solutions.
 - **Remote Work Relationships:** Remote work relationships are an important part of the present work culture and refer to professional relationships between individuals working from different locations. These relationships are possible through digital communication tools and involve virtual collaboration, flexible schedules, and a dependence on technology.
-

3.0. Chapter Three: Evaluation on Field Attachment

The evaluation on field attachment derives in depth work flow from introduction of backend programming unto the coding, debugging and testing an internship mini project.

3.0.1 Introduction to backend Programming

Backend development is the skill attained for one to design, development, test and implement software applications. It requires for one to undertake specific tools and mechanism of frameworks and therefore, during this internship were guided through the following requirements; Structured Query Language (SQL), Spring Boot Framework, DataGrip, IntelliJ, Application Programming Interface and Java Language and Postman

3.0.2 Structured Query Language

According to Tpoint Tech (2025) , SQL is Structured Query Language used to manage data in a relational database system. This tag refers explicitly to the ISO/ANSI SQL standard; not to any specific implementation of that standard. Structured Query Language (SQL) is a special-purpose programming language designed for managing data held in a Relational Database Management System (RDBMS). SQL-like languages can also be used in Relational Data Stream Management Systems (RDSMS), or in "not-only SQL"(NoSQL) databases. SQL comprises of 3 major sub-languages:

- Data Definition Language (DDL): to create and modify the structure of the database;
- Data Manipulation Language (DML): to perform Read, Insert, Update and Delete operations on the data of the database;
- Data Control Language (DCL): to control the access of the data stored in the database.

The core DML operations are Create, Read, Update and Delete (CRUD for short) which are performed by the statements INSERT, SELECT, UPDATE and DELETE.

There is also a (recently added) MERGE statement which can perform all 3 write operations (INSERT, UPDATE, DELETE).

3.0.3 Java Language

According to Hamedani (2024), Java is a robust and versatile language, perfect for backend development. Known for its performance and scalability, Java is a top choice for building large-scale, enterprise-level applications.

According to Hamedani (2024), Essential Concepts are:

- Basics: Variables, primitive and reference types, constants, casting, arrays, arithmetic expressions
- Control Flow: Comparison operators, logical operators, ternary operator, if/else, switch/case, for, foreach, while, do..while, break, continue
- Object-oriented Programming: Classes, objects, constructors, method overloading, static members, inheritance, access modifiers, method overriding, upcasting/downcasting, polymorphism, abstract classes, final classes, interfaces
- Exceptions: Exceptions hierarchy, try/catch/finally, throw, custom exceptions
- Generics: Generic classes/methods/interfaces, constraints, type erasure, type parameters
- Collections Framework: Iterable, Iterator, Collection, List, Comparable, Comparator, Queue, Set, Map
- Functional interfaces: Lambda expressions, Consumer, Supplier, Function, Predicate, BinaryOperator, UnaryOperator

3.0.4 Spring Boot Framework

According to Walls, C. (2016), Spring is a framework and is written in the Java language. It provides lots of modules, such as Spring Data, Spring Security, Spring Cloud, Spring Web, and so on. It is popular for building enterprise applications. Initially, it was looked at as a Java Enterprise Edition (EE) alternative. Spring supports Dependency Injection (Inversion of Control) and Aspect-Oriented Programming out of the box as its core. Apart from Java, Spring also supports other JVM languages such as Groovy and Kotlin. The principle and design patterns – Inversion of Control (IoC), Dependency Injection (DI), and Aspect-Oriented Programming (AOP).

According to Walls, C. (2016), Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can “just run”. It takes an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration. You can use Spring Boot to create Java applications that can be started using `java -jar` or more traditional war deployments. It also provides a command line tool that runs “spring scripts”. primary goals are:

- Provide a radically faster and widely accessible getting started experience for all Spring development.
 - Be opinionated out of the box, but get out of the way quickly as requirements start to diverge from the defaults.
 - Provide a range of non-functional features that are common to large classes of projects (e.g., embedded servers, security, metrics, health checks, externalized configuration).
 - Absolutely no code generation and no requirement for XML configuration.
-

According to Walls, C. (2016), Spring Boot brings a great deal of magic to Spring application development. But

there are four core tricks that it performs:

- Automatic configuration—Spring Boot can automatically provide configuration for application functionality common to many Spring applications.
- Starter dependencies—You tell Spring Boot what kind of functionality you need, and it will ensure that the libraries needed are added to the build.
- The command-line interface—This optional feature of Spring Boot lets you write complete applications with just application code, but no need for a traditional project build.
- The Actuator—Gives insight into what's going on inside of a running Spring Boot application.

Spring Boot doesn't implement any enterprise Java specifications such as JPA or JMS. It does support several enterprise Java specifications, but it does so by automatically configuring beans in Spring that support those features. For instance, Spring Boot doesn't implement JPA, but it does support JPA by auto-configuring the appropriate beans for a JPA implementation (such as Hibernate).

Finally, Spring Boot doesn't employ any form of code generation to accomplish its magic. Instead, it leverages conditional configuration features from Spring 4, along with transitive dependency resolution offered by Maven and Gradle, to automatically configure beans in the Spring application context.

3.0.5 Postman

Postman is an Application Programming Interface (API) testing tool. API acts like an interface between a couple of applications and establishes a connection between them (GeeksforGeeks, 2024).

An API is a collection of agreements, functions, and tools that an application can provide to its users for successful communication with another application. We require an API whenever we access an application like checking news over the phone, Facebook (GeeksforGeeks, 2024).

According GeeksforGeeks, (2024), Postman was designed in the year 2012 by software developer and entrepreneur Abhinav Asthana to make API development and testing straightforward. It is a tool for testing the software of an API. It can be used to design, document, verify, create, and change APIs.

Postman has the feature of sending and observing the Hypertext Transfer Protocol (HTTP) requests and responses. It has a graphical user interface (GUI) and can be used in platforms like Linux, Windows and Mac. It can build multiple HTTP requests – POST, PUT, GET, PATCH and translate them to code.

- Postman has a huge user base and has become a very popular tool because of the reasons listed below:
 - Postman comes without any licensing cost and is suitable for use for the teams with any capacity. It can be installed as a standalone application.
 - Postman allows easy maintenance of test suites with the help of collections. Users can make a collection of API calls which can have varied requests and sub-folders.
 - Postman is capable of building multiple API calls like SOAP, REST, and HTTP.
 - Postman can be used for test development by addition of checkpoints to HTTP response codes and other parameters.
 - Postman can be integrated with the continuous integration and either continuous delivery or continuous deployment (CI/CD) pipeline.
-

- Postman has big community support. The Postman console allows debugging test steps.
- Postman gives the option to import/export Environments and Collections, enabling easy sharing of files.

Postman can be installed in operating systems like Mac, Windows and Linux. It is basically an independent application which can be installed in the following ways:

- Postman can be installed from the Chrome Extension (will be available only in Chrome browser).

3.0.6 DataGrip

According to RisingWave (2024), DataGrip is an IDE designed to make the process of analysing any amount of data easier and more efficient. It enables users to access multiple databases in one interface, streamlining the process of analysing large amounts of data quickly and accurately. It also provides features that allow developers to create applications faster with less code, making it easy for any user regardless of their experience level to get started quickly and accurately. It is designed to simplify this process, making it easier and more efficient for data scientists, analysts, and anyone working with data.

According to RisingWave (2024), DataGrip is a database IDE tailored to suit the specific needs of professional SQL developers. Widely used by Enterprises, it simplifies the process of analysing large amounts of data quickly and accurately. This versatile tool supports various databases like MySQL, PostgreSQL, SQL Server, Oracle, and more. With features such as a database explorer, SQL editor, and query builder, it provides intelligent coding assistance for editing SQL code efficiently. With an intuitive editor offering intelligent coding assistance, including auto-completion and query profiling capabilities, it enhances the efficiency and accuracy of SQL code editing.

The purposes of DataGrip (RisingWave 2024):

- DataGrip stands out as a comprehensive IDE designed to streamline database management tasks for SQL developers. Its primary objective is to provide a unified platform for accessing and manipulating various databases seamlessly. By offering advanced features like context-sensitive code completion and query execution modes, empowers users to work with multiple databases efficiently.
- The strength of DataGrip lies in its extensive support for numerous databases, ensuring compatibility with popular systems like MySQL, PostgreSQL, Oracle, and more. This broad database coverage enables SQL developers to work across different platforms without encountering compatibility issues or limitations.
- The standout features are its multi-engine support, allowing users to connect to diverse database systems seamlessly. Whether it's MySQL, PostgreSQL, or Microsoft SQL Server, facilitates easy integration with various engines through its intuitive interface.

The key features of DataGrip:

- The Context-Sensitive Code Completion feature is a game-changer for developers. By analysing the context of the code being written, intelligently suggests relevant code snippets, table names, and even column names. This functionality significantly accelerates the coding process by reducing manual input errors and streamlining the development workflow. For developers, the benefits are immense. It not only saves

time by offering quick suggestions but also improves code accuracy by minimizing typos and syntax errors

- Developers have the flexibility to execute queries in various modes tailored to their specific needs. Whether it's running queries for data retrieval, modification, or analysis, provides different execution modes to optimize performance and enhance result accuracy. By understanding these modes, developers can choose the most suitable option based on their query requirements.
- The SQL Log feature acts as a comprehensive audit trail for all executed queries. By maintaining a detailed log of query history, developers can track changes made to databases over time, identify performance bottlenecks, and troubleshoot issues effectively. This visibility into query execution is crucial for maintaining database integrity and optimizing system performance.

3.0.7 IntelliJ

According to TechSpot (2025), IntelliJ IDE is a powerful integrated development environment primarily designed for Java development with extensive support for other languages and frameworks. IntelliJ IDEA comes with one of the most powerful code editors in the industry. It understands the ins and outs of your code thanks to initial indexing, which allows it to detect errors on the fly, suggest code completion options with precise context awareness, perform safe refactoring, and much more. IntelliJ IDEA uses machine learning to ensure the most relevant suggestion is at the top of the list.

Features (TechSpot, 2025):

- Full line code completion: powered by an advanced deep learning model, this feature enhances coding efficiency by predicting and suggesting entire line of code based on contextual analysis. It runs entirely on your device, ensuring your data remains secure without any external server communication.
 - AI Assistant General Availability: AI Assistant comes with useful functionality that can simplify your daily tasks. It offers integrated AI chat can do things like automatically write documentation comments for you, suggest names, generate commit messages, and more.
 - Language Injections: IntelliJ IDEA lets you work with elements of code in other languages embedded in your code. Won't notice a difference between your experience working with the core Java code and experience with the code that you have injected. Provide code highlighting, completion, error detection, and other coding assistance features for embedded code fragments.
 - Intelligent code completion: Provides context sensitive code completion, offering suggestions that are only valid for the current position. Don't need to use any shortcuts or configure additional settings; code completion will start as soon as you begin typing your code in the editor.
-

3.0.9 Restful API

Representational state transfer (REST) is a software architectural style that defines a set of constraints to be used for creating Web services. RESTful Web services allow the requesting systems to access and manipulate textual representations of Web resources by using a uniform and predefined set of stateless operations (Kandwal, 2020).

REST is an acronym for REpresentational State Transfer and an architectural style for distributed hypermedia systems (Gupta, 2025).

According to Gupta (2025), REST is not a protocol or a standard, it is an architectural style. During the development phase, API developers can implement REST in a variety of ways. REST also has its guiding principles and constraints. These principles must be satisfied if a service interface is to be referred to as RESTful. A Web API (or Web Service) conforming to the REST architectural style is called a REST API (or RESTful API).

REST is based on some constraints and principles that promote simplicity, scalability, and statelessness in the design. The six guiding principles or constraints of the RESTful architecture are:

- Uniform Interface: By applying the principle of generality to the components interface, we can simplify the overall system architecture and improve the visibility of interactions. Multiple architectural constraints help in obtaining a uniform interface and guiding the behaviors of components.
- Client-Server: The client-server design pattern enforces the **separation of concerns**, which helps the client and the server components evolve independently.
- Stateless: This mandates that each request from the client to the server must contain all of the information necessary to understand and complete the request.
- Cacheable: The cacheable constraint requires that a response should implicitly or explicitly label itself as cacheable or non-cacheable.
- Layered System: The layered system style allows an architecture to be composed of hierarchical layers by constraining component behaviors. In a layered system, each component cannot see beyond the immediate layer they are interacting with.
- Code on Demand: REST also allows client functionality to be extended by downloading and executing code in the form of applets or scripts.

According to Gupta (2025), The key abstraction of information in REST is a resource. Any information that we can name can be a resource. For example, a REST resource can be a document or image, a temporal service, a collection of other resources, or a non-virtual object (e.g., a person). The state of the resource at any particular time is known as the resource representation. The resource representations consist of the data, the meta data describing the data and the hypermedia links that can help the clients transition to the next desired state.

3.1. Level of Accomplishment of duties and responsibilities assigned

LEVEL OF ACCOMPLISHMENT OF DUTIES	RESPONSIBILITIES ASSIGNED
Introduction to Java backend programming	• Reading to understand • Background concept review
Introduction to Structured Query Language (SQL)	• Hands on practical script writing • Brief and precise explaining of concept
Introduction to Data Grip	• Download and Installation • Writing SQL script on Console
Introduction to Microsoft Teams	• Download and Installation • Account creating
Introduction to Object Oriented Programming	• Review on Object Oriented Programming concepts • Precise concept explanation
Introduction to Spring Boot	• Write Spring Boot projects in IntelliJ • Understanding Spring Boot framework
Introduction to IntelliJ	• Writing source code in Java language • Write Spring Boot projects with Java
Introduction to Application Programming Interface (API)	• Consuming Application Programming Interface (API)

3.2. New knowledge and skills gained in each of the duties and responsibilities

LEVEL OF ACCOMPLISHMENT OF DUTIES	RESPONSIBILITIES ASSIGNED	NEW KNOWLEDGE	SKILLS GAINED
Introduction to Java backend programming	➤ Reading to understand ➤ Background concept review	➤ Depth in Java backed programming	➤ Java backend programming
Introduction to Structured Query Language (SQL)	➤ Hands on practical script writing ➤ Brief and precise explaining of concept	➤ SQL concepts ➤ Join Concepts ➤ View concept	➤ Writing scripts of single tables ➤ Writing scripts of multiple tables
Introduction to Data Grip	➤ Download and Installation ➤ Writing SQL script on Console	➤ Data Grip tool ➤ Console interfacing	➤ Designing databases ➤ Script understanding
Introduction to Microsoft Teams	➤ Download and Installation ➤ Account creating	➤ Microsoft Teams software interacting	➤ Understanding Microsoft Teams
Introduction to Object Oriented Programming	➤ Review on Object Oriented Programming concepts ➤ Precise concept explanation	➤ Classes and Objects ➤ Inheritance and Interfaces ➤ Exceptions	➤ Source code writing ➤ Language understanding ➤ Brief and precise explanation
Introduction to Spring Boot	➤ Write Spring Boot projects in IntelliJ ➤ Understanding Spring Boot framework	➤ Spring Boot framework ➤ Spring Annotation ➤ Spring JdbcTemplate	➤ Incite understanding of Spring Boot ➤ Rapid Application Development
Introduction to IntelliJ	➤ Writing source code in Java language	➤ Java SDK integration	➤ In-depth understanding

	➤ Write Spring Boot projects with Java	➤ IntelliJ structure ➤ Spring Boot structure	➤ Project development ➤ Project structuring
Introduction to Application Programming Interface (API)	➤ Consuming Application Programming Interface (API)	➤ Consuming internal Application Programming Interface (API) ➤ Consuming external Application Programming Interface	➤ Writing program to Consume Application Programming Interface (API)

3.3. Most interesting experiences

1. Learning Java backend programming.
2. Learning from experienced programming team.
3. Hands on work doing.
4. Open minded experiencing.
5. Supervisor to Internee communication.
6. Mentoring and guidance from various department.
7. Free minded expression.
8. High co-operation team work.
9. Open willingness of every member.
10. High professional skills.

3.4. Relatedness of University's taught programmes to the Field of work

Below are the University's taught programs:

1. Java language programming.
 2. Communication skills.
 3. Problem solving.
 4. Ethics and integrity.
 5. Database Management.
 6. Project Management and development.
-

3.5. Challenges faced and how managed - (both works related & organisational factors & from an individual perspective)

Work related challenges	How managed
Task completion	By completing particular task assigned.
Task design and development.	By reading and researching.
Misunderstanding tasks.	By consulting the supervisor.
Failure to understand tasks.	By asking assistance and explanation from supervisor.

Organizational challenges	How managed
Time management.	By reporting with regulated time frame.
Cooperation with others.	By creating a friendly environment.
Understanding organizational structure.	By communicating with others even when from other departments.

Individual perspective challenges	How managed
Naïve in communication	By daily communication practice.
Naïve open expression	By daily open-minded practice.
Poor explaining concepts	By regular practices for brief and precise explanations
Hygiene	By practicing company hygiene environment
Time management	By making sure report with early time range.
Project work making	By making sure completion of particular task.

3.6. Benefits derived from Field Attachment

- Learning Java backend programming.
 - Learning from experienced programming team.
 - Hands on work doing.
 - Open minded experience.
 - Supervisor to Internee communication.
 - Mentoring and guidance from various department.
 - Free minded expression.
 - High co-operation team work.
 - Open willingness of every member.
 - High professional skills.
-

- Improved programming skills.
- Good communication skills.

3.7. Adequacy in University's preparing the student for Field Attachment

3.8. Preparedness of the Agency to receive and manage Students for Field Attachment

According Parkes, M., Stein, S. and Reading, C. (2015), Field Attachment preparedness from the perspectives of university teachers (staff) and students across different disciplines and institutions. The term Field Attachment preparedness is complex and involves several dimensions, such as 'being informed, making the right choices, having realistic expectations and being motivated'. Some scholars argue that students enter internship with weaker basic disciplinary skills than needed (for example, statistical skills for psychology studies), while others argue that students show weak generic writing and reading skills). Still others point to the importance of behavioural characteristics, such as academic perseverance.

According Parkes, M., Stein, S. and Reading, C. (2015), Preparedness is conceptualised as an overarching term comprising reading and writing skills, basic disciplinary skills and academic working skills, all of them preconditions for further academic development. The focus is on students' and university teachers' experiences related to these dimensions. Academic staff are crucial for student learning but are also guardians of quality in their disciplines, setting the standards for what counts as good performance. Students are also important informants as they are in a process of grasping what is required from them in their formative time as students in for Field Attachment.

3.9 Career Motivation

Career motivation is the desire to exert effort to enhance career goals (IResearchNet, n.d.). It is a multidimensional construct that combines elements of needs, interests, and personality characteristics that reflect the stimulus, direction, and persistence of career-related behaviours. According to IResearchNet (n.d.), Career motivation is organized into three domains. Career insight is the stimulus or energizing component. This is people's ability to be realistic about themselves and their careers. People who are high in career insight have an accurate understanding of their strengths and weaknesses and set clear career goals as below;

- Understand the deeper purpose behind your work.
- Reflect on how your role contributes to your personal goals or helps others.
- Specific, Measurable, Achievable, Relevant, Time-bound.
- Break big goals into smaller milestones to track progress and stay focused.
- Acknowledge every achievement, no matter how minor.
- Take courses, attend workshops, or explore new tools.
- Continuous growth keeps your work fresh and boosts confidence.
- Surround yourself with mentors, peers, and professionals who uplift you.
- Share goals and challenges—it helps you stay accountable and inspired.
- Focus on what's going well rather than what's lacking.
- Keep a gratitude or success journal to reflect on progress.
- Try new approaches to routine tasks.
- Creativity can reignite excitement and help you see things from a fresh perspective.
- Avoid burnout by setting boundaries and taking breaks.
- Recharge regularly to return with renewed energy.

4.0. Chapter Four: Conclusions and Recommendations

1. Conclusions - emphasise your important points in the report including strengths and weaknesses
2. Recommendations - This part should include recommendations to strengthen the Field Attachment/ internship programme as part of the curriculum at Makerere University; as well as for improvement of service delivery at your place of attachment. Recommendations can also focus on advising other students for future Internship/ Field Attachment.

4. REFERENCES

Schildt, H. (2014) **Java: The Complete Reference**. 9th edn. New York: McGraw-Hill Education.

PostgreSQL Global Development Group (2023) **PostgreSQL Documentation**. Available at: <https://www.postgresql.org/docs/> (Accessed: 20 July 2025).

Walls, C. (2016) **Spring Boot in Action**. Shelter Island, NY: Manning Publications.

Tpoint Tech (2025) **SQL Tutorial: Learn SQL for Beginners and Professionals**. Available at: <https://www.tpointtech.com/sql-tutorial> (Accessed: 24 July 2025).

Canepa, G. (2016) **PostgreSQL Database Handbook**. Magdeburg: University of Magdeburg. Available at: <http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf> (Accessed: 24 July 2025).

Sharma, S. (2021) **Modern API Development with Spring and Spring Boot**. Birmingham–Mumbai: Packt Publishing. Available at: https://download.bibis.ir/Books/Programming/java/2021/Modern-API-Development-with-Spring-and-Spring-Boot_by-Sourabh-Sharma_bibis.ir.pdf (Accessed: 24 July 2025).

GeeksforGeeks (2024) **Postman Tutorial**. Available at: <https://www.geeksforgeeks.org/software-testing/postman-tutorial/> (Accessed: 24 July 2025).

W3Schools (2025) **SQL Tutorial**. Available at: <https://www.w3schools.com/sql/> (Accessed: 24 July 2025).

GoalKicker (2018) **Spring Framework Notes for Professionals**. Compiled from Stack Overflow Documentation. Available at: <https://goalkicker.com/SpringFrameworkBook/SpringFrameworkNotesForProfessionals.pdf> (Accessed: 24 July 2025).

Spring.io (2025) **Building REST Services with Spring**. Available at: <https://spring.io/guides/tutorials/rest> (Accessed: 24 July 2025).

Chacon, S. and Straub, B. (2014) **Pro Git**. 2nd edn. New York: Apress. Available at: <https://www.dbooks.org/pro-git-1484200764/> (Accessed: 18 July 2025).

Parkes, M., Stein, S. and Reading, C. (2015) ‘Student preparedness for university e-learning environments’, **The Internet and Higher Education**, 25, pp. 1–10. Available at: https://www.researchgate.net/publication/268691014_Student_preparedness_for_university_e-learning_environments (Accessed: 15 July 2025).

DataCamp (2024) *GIT Push and Pull Tutorial*. Available at: <https://www.datacamp.com/tutorial/git-push-pull> (Accessed: 20 July 2025).

KeyDifferences (no date) *Difference Between Duties and Responsibilities*. Available at: <https://keydifferences.com/difference-between-duties-and-responsibilities.html> (Accessed: 15 July 2025).

restfulapi.net, 2024. *REST API Tutorial*. [online] Available at: <https://restfulapi.net/> [Accessed 25 Jul. 2025].

Kandwal, V., 2020. *REST API Tutorial – REST Client, REST Service, and API Calls Explained with Code Examples*. [online] freeCodeCamp. Available at: <https://www.freecodecamp.org/news/rest-api-tutorial-rest-client-rest-service-and-api-calls-explained-with-code-examples/> [Accessed 25 Jul. 2025].

Gupta, L., 2025. *What is REST? REST API Tutorial*. [online] restfulapi.net. Available at: <https://restfulapi.net/> [Accessed 25 Jul. 2025].

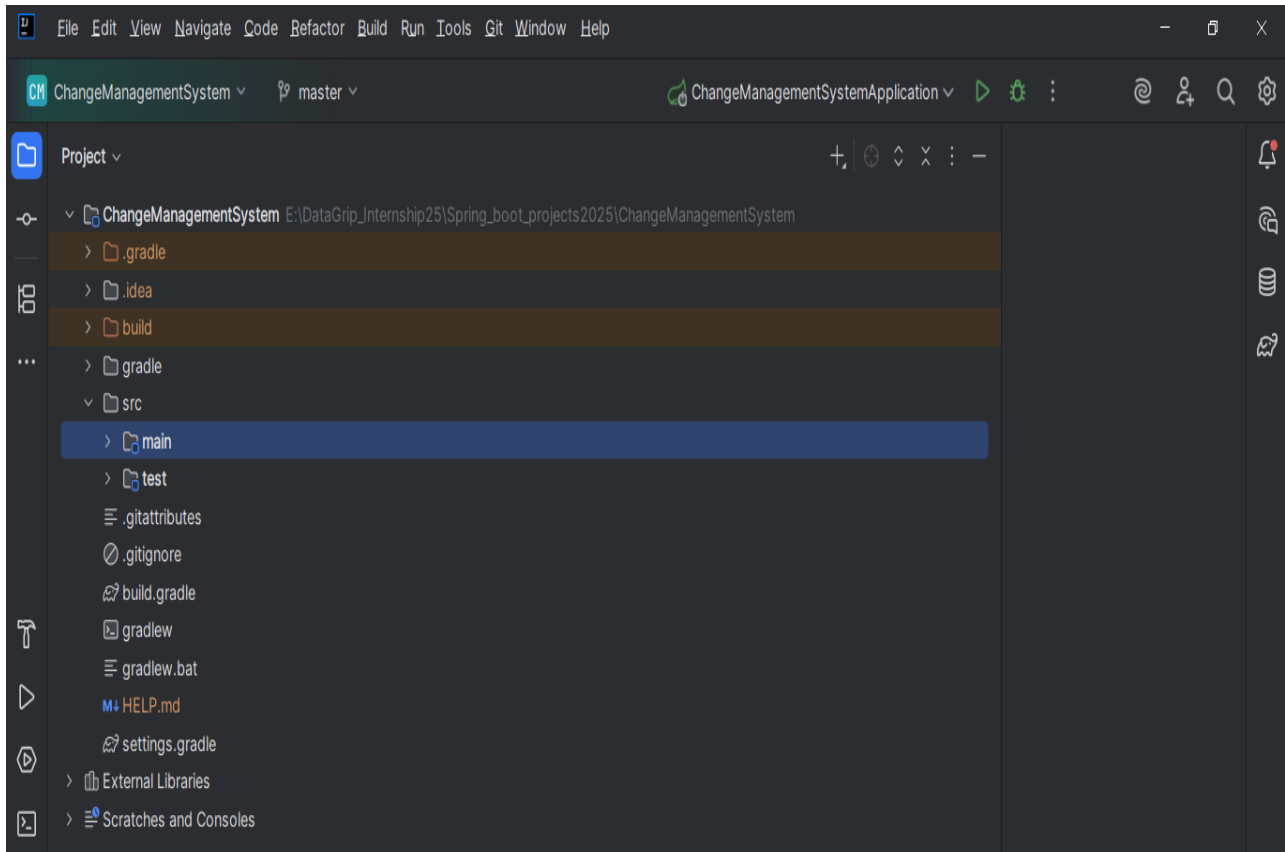
RisingWave, 2024. *Datagrip: Your Ultimate Guide to Database Management*. [online] RisingWave. Available at: <https://risingwave.com/blog/datagrip-demystified-a-comprehensive-overview/> [Accessed 25 Jul. 2025].

IResearchNet. (n.d.) *Career motivation*. [online] Career Development. Available at: <https://career.iresearchnet.com/career-development/career-motivation/> [Accessed 25 Jul. 2025].

5 APPEDICES

INTERSHIP MINI PROJECT: CHANGE REQUEST MANAGEMENT SYSTEM

The image below shows the mini project structure. The project was written in Java language using Spring Boot Framework structure and an IntelliJ tool.



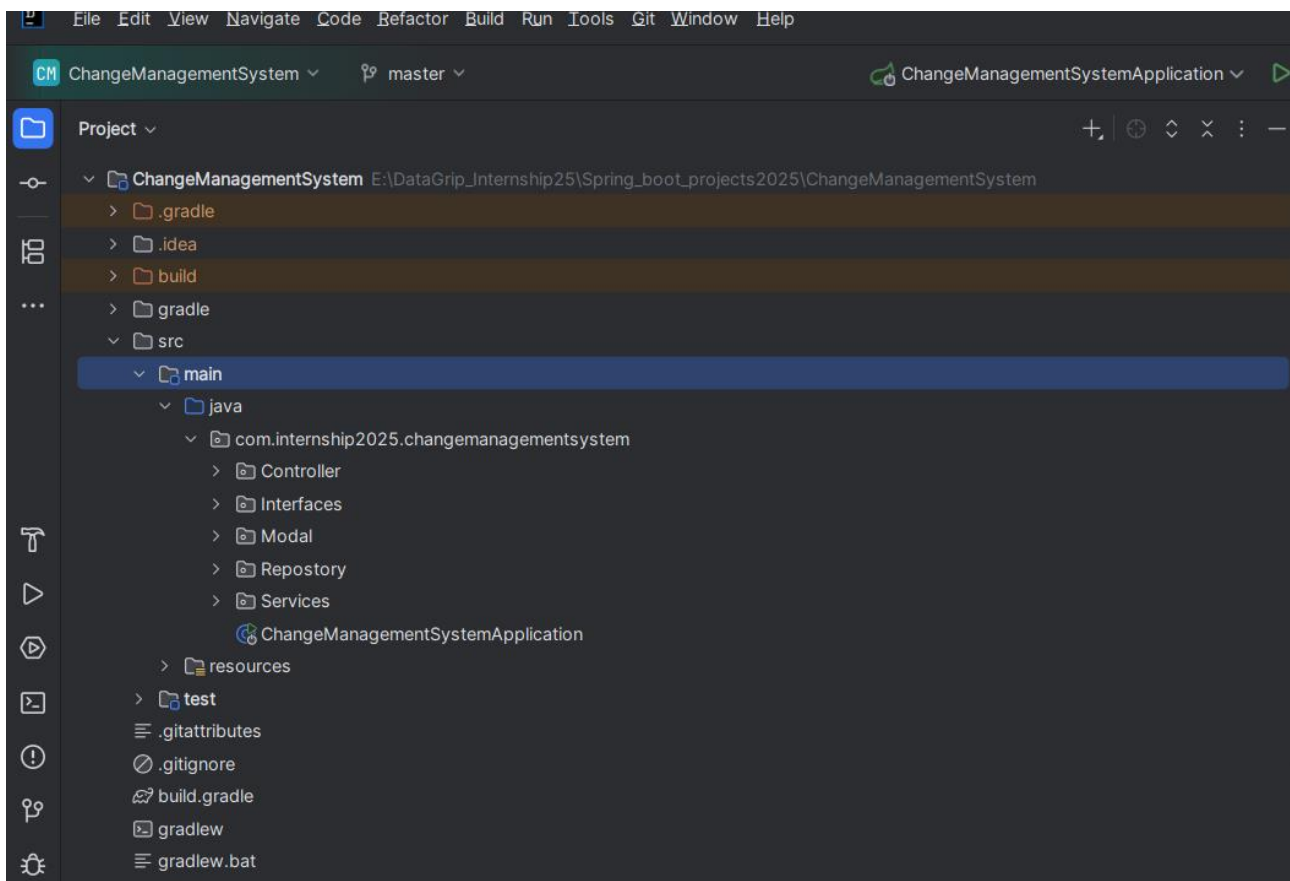
The diagram below shows the source (src) folder structure in which the main folder comprises of the section of creating modals, Java classes. The various folders in the main are as follows:

Controller: Contains Java classes that encapsulates the mapping of Http Request to methods of response class.

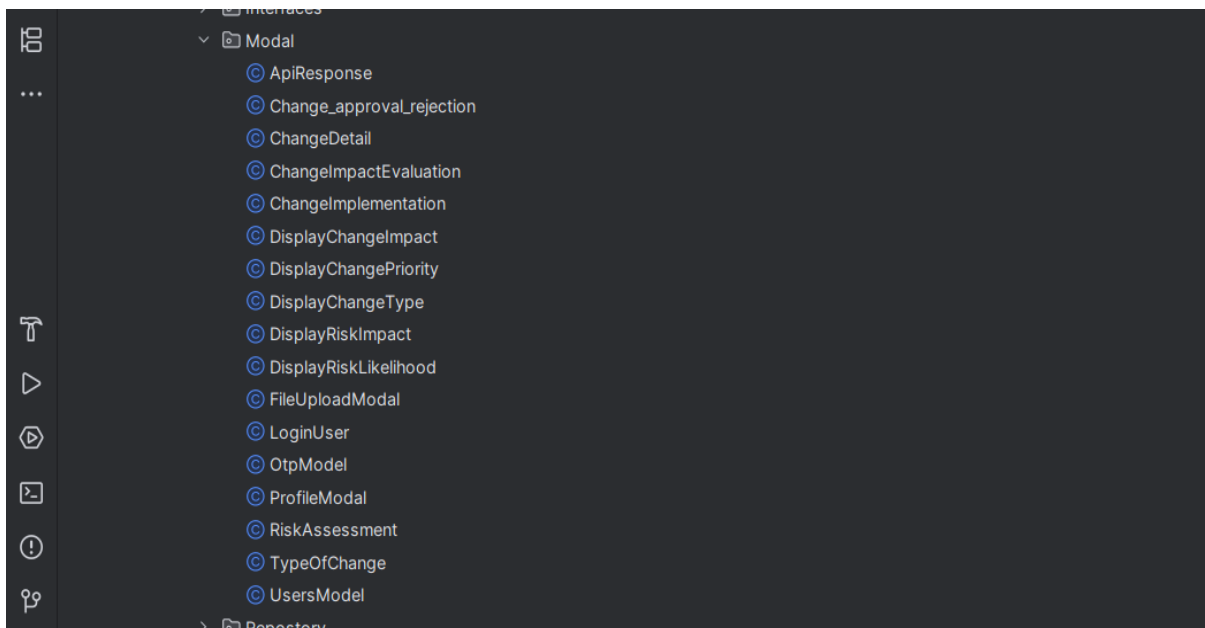
Modal: This is for create plain Old Java Objects.

Repository: This folder is for creating classes that connect or linkup to the database archives using the JdbcTemplate of the JDBC API.

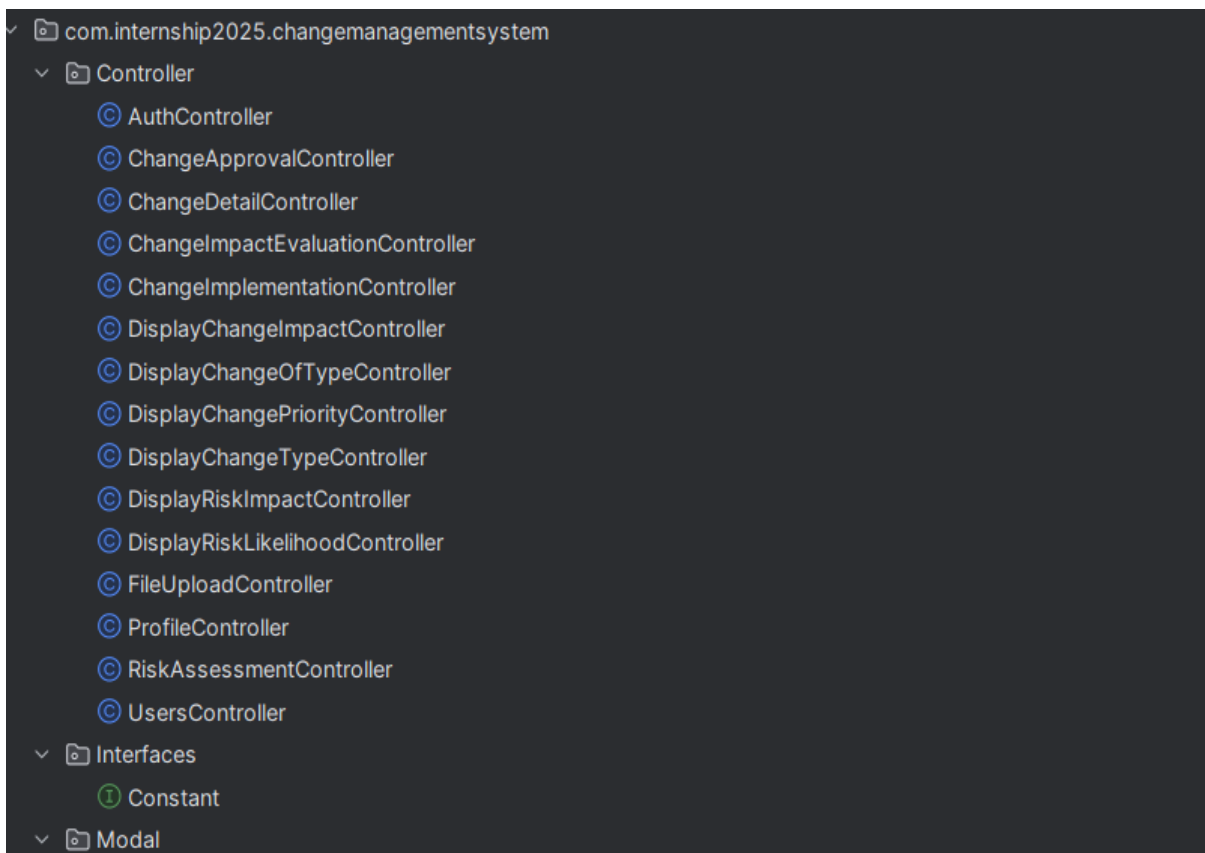
Service: This is for creating classes for constituting business logic writing.



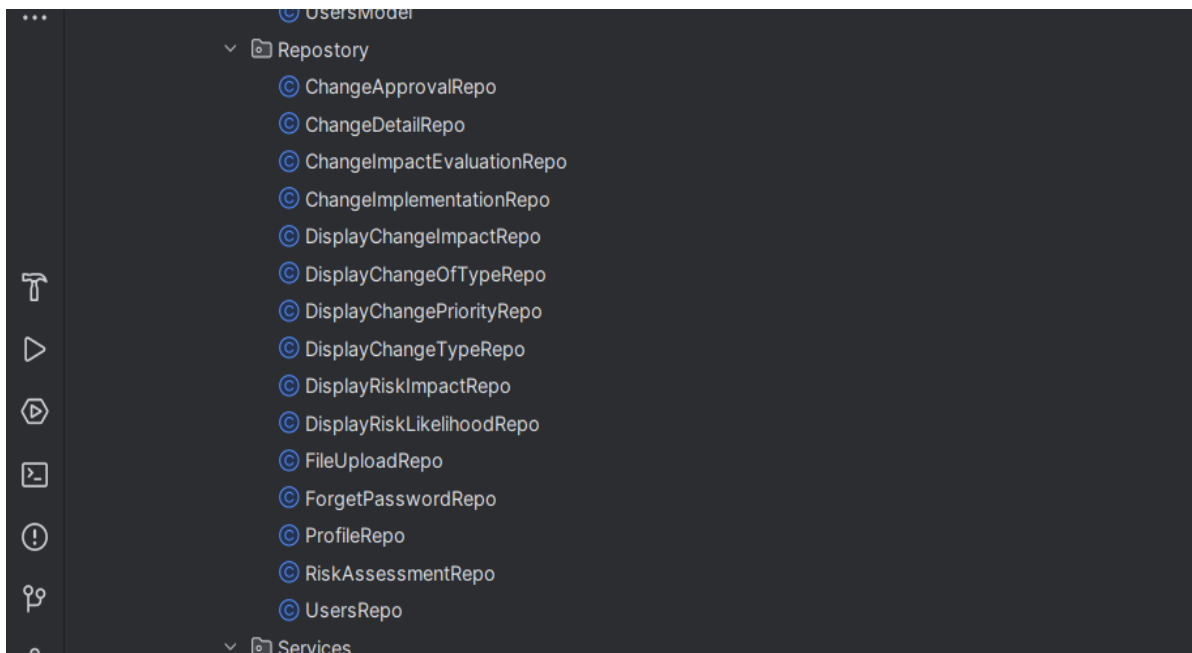
The Modal Folder



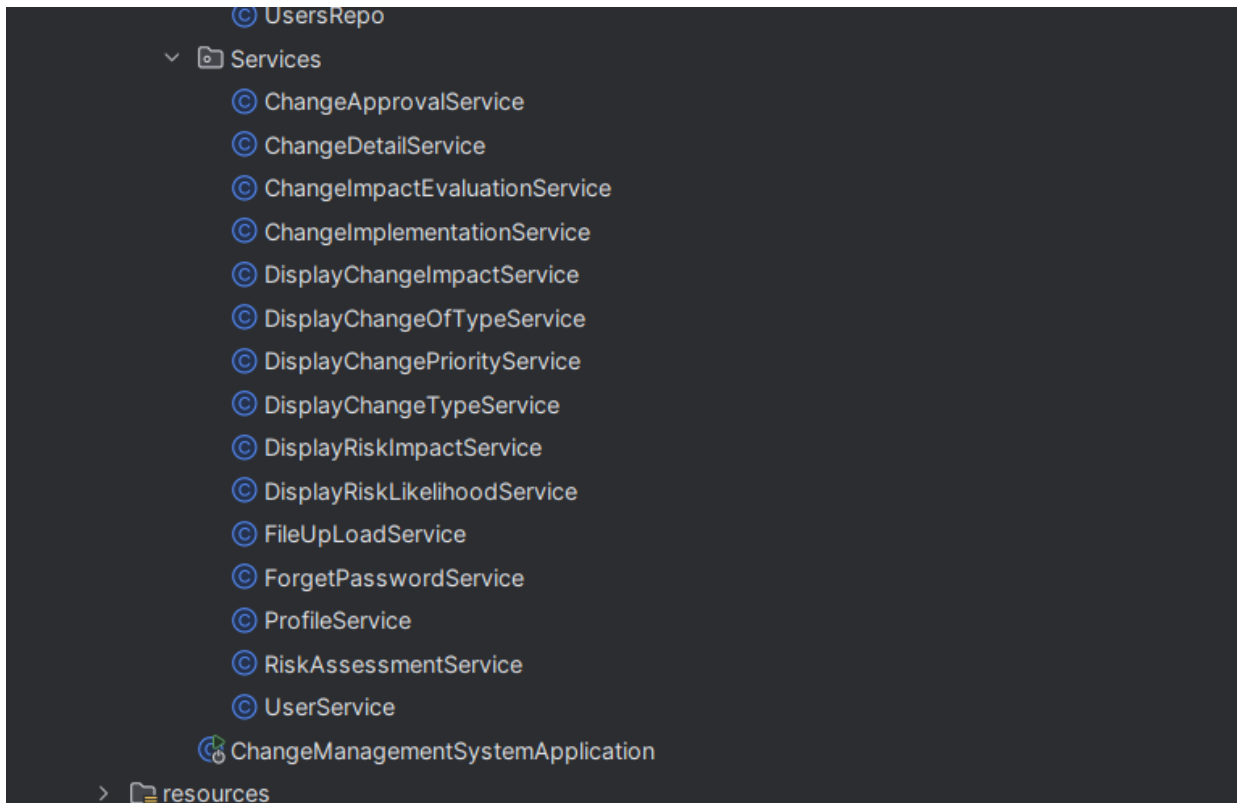
The Controller and Interface Folder



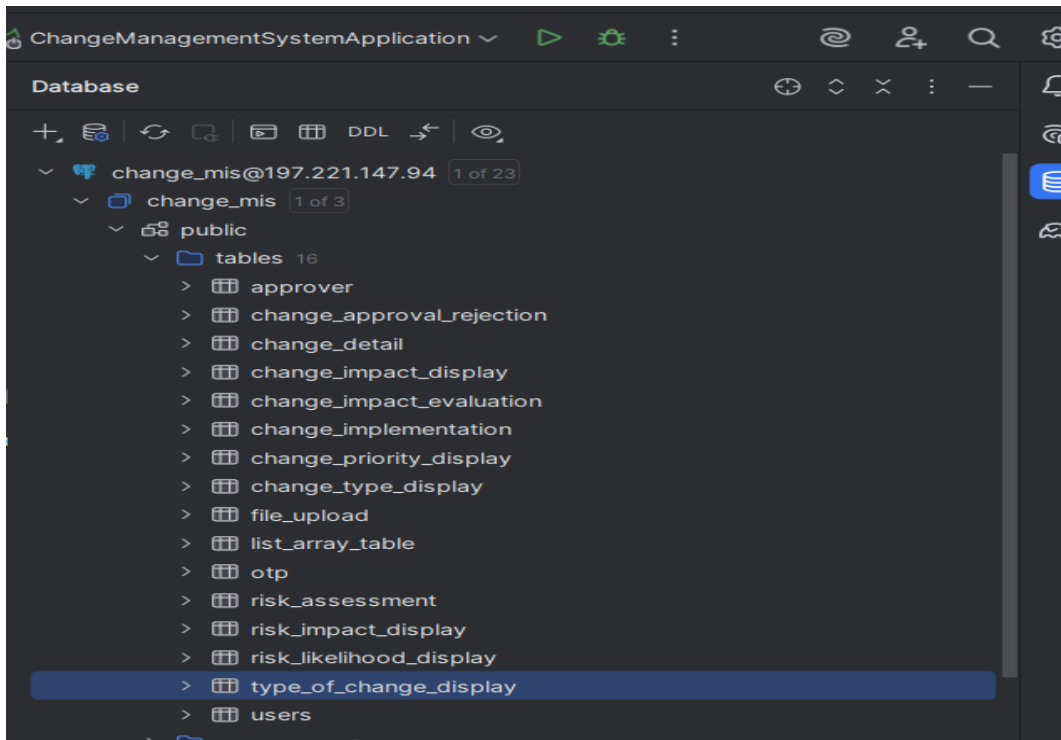
Repository Folder



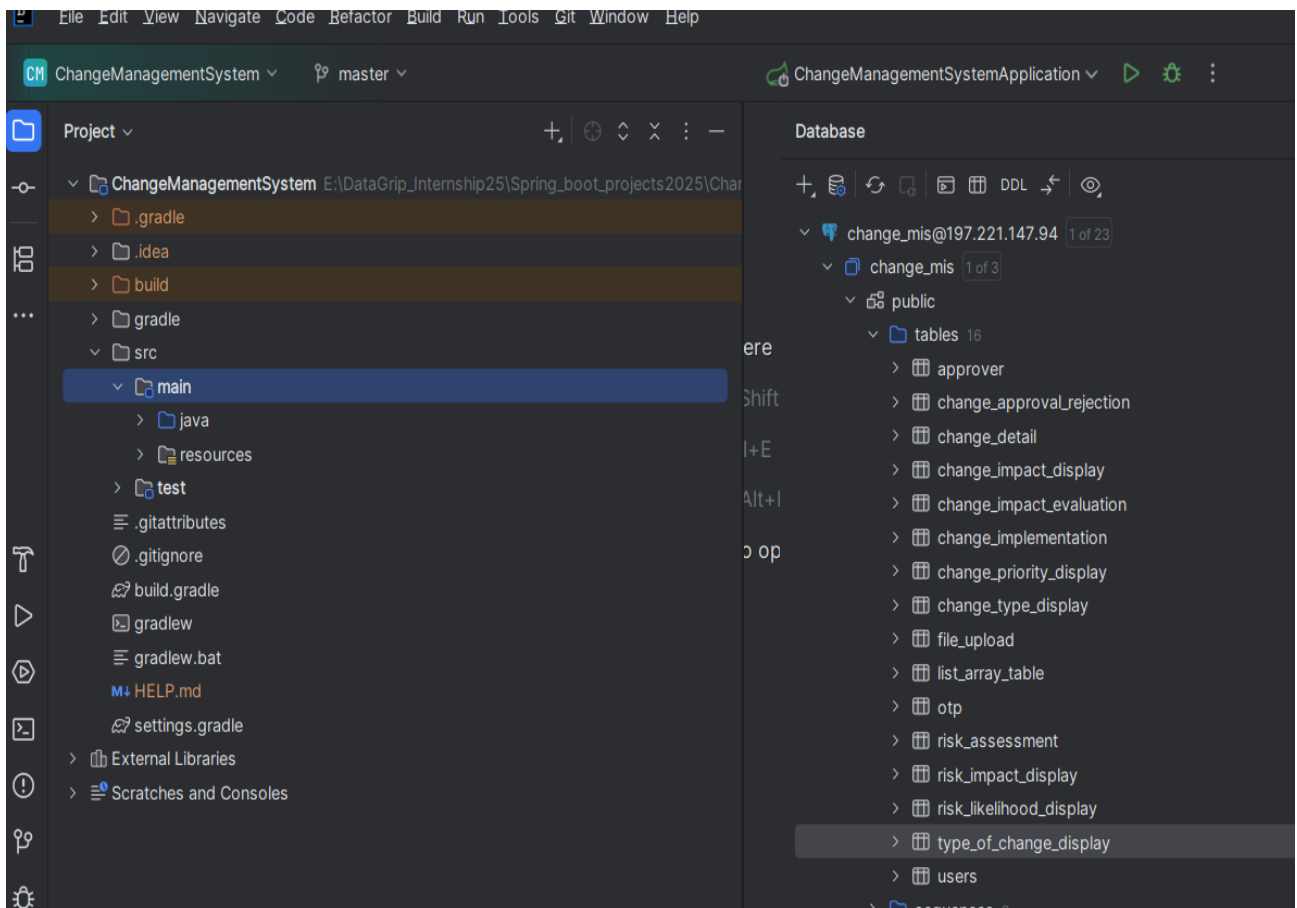
The Service Folder



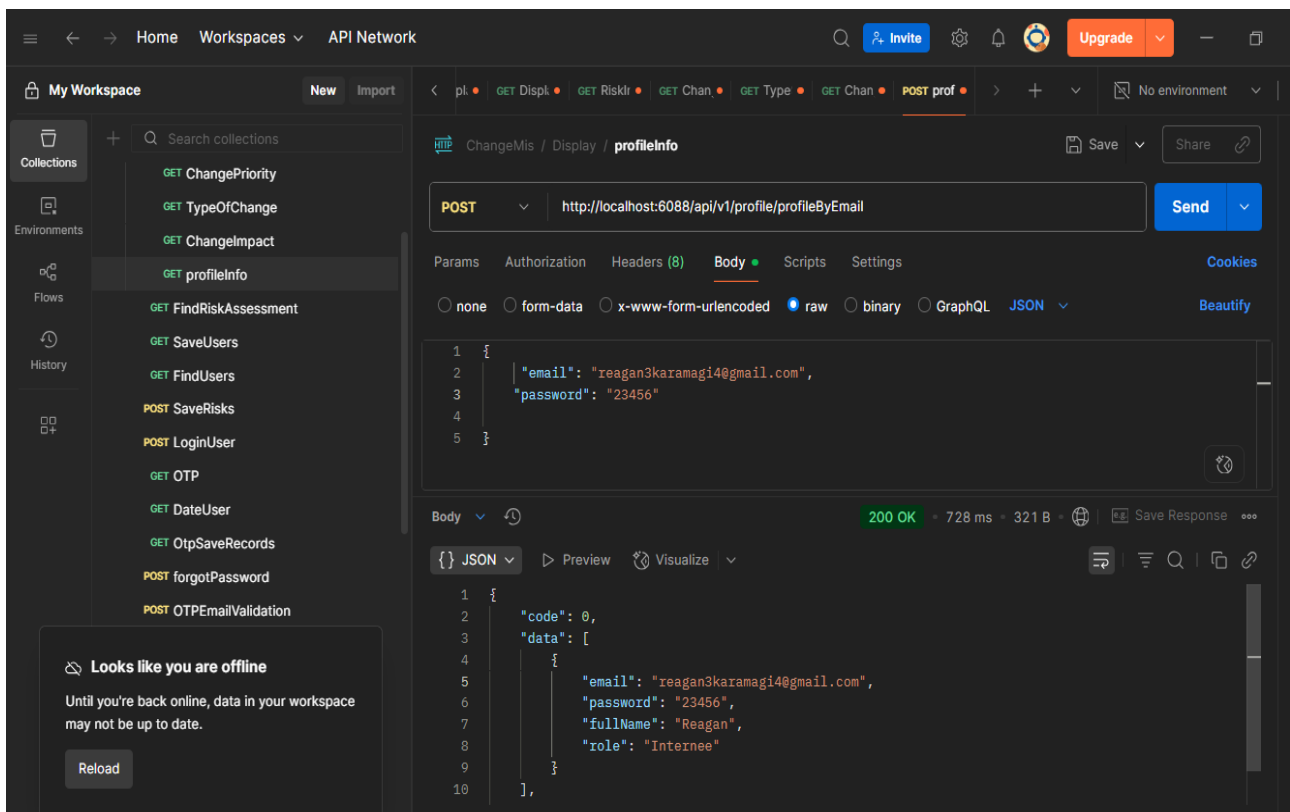
CHANGE MANAGEMENT DATABASE



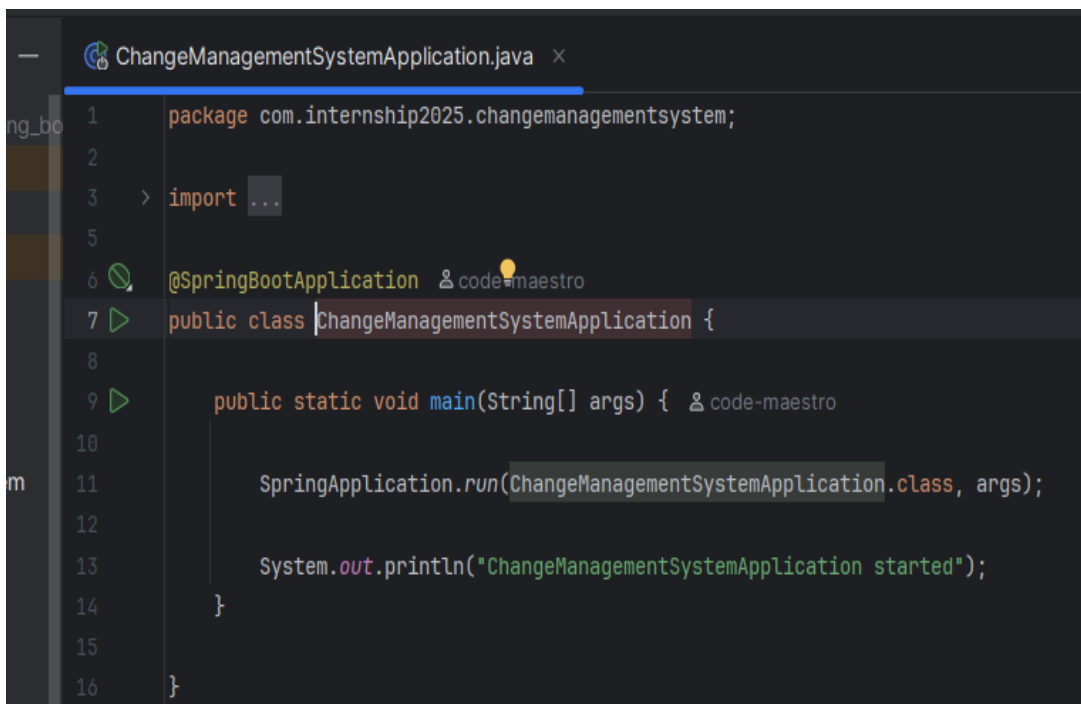
The Project and Database Structure



Postman Structure



The Project Main Class



The User Repository Source code

```
ChangeManagementSystemApplication.java  © UsersRepo.java x

package com.internship2025.changemanagementsystem.Repository;

import com.internship2025.changemanagementsystem.Model.LoginUser;
import com.internship2025.changemanagementsystem.Model.UsersModel;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public class UsersRepo {

    private final JdbcTemplate jdbcTemplate;

    public UsersRepo(JdbcTemplate jdbcTemplate) { this.jdbcTemplate = jdbcTemplate; }

    public boolean save(UsersModel user) {

        String sql = "insert into users(full_name, email, password, confirm_password, role, phone) values (?, ?, ?, ?, ?, ?)";

        return jdbcTemplate.update(sql, user.getFullName(), user.getEmail(), user.getPassword(),
            user.getConfirmPassword(), user.getRole(), user.getPhone()) > 0;
    }
}
```

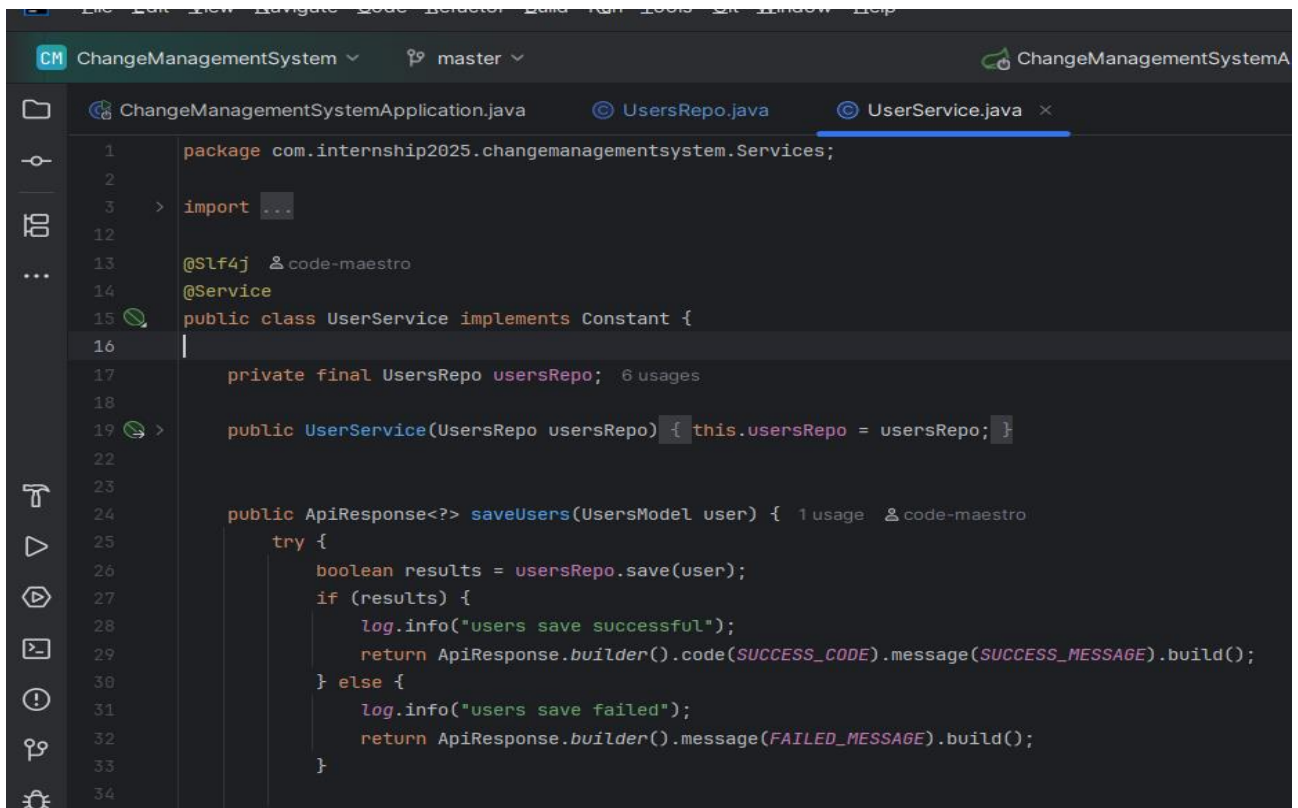
```
public List<LoginUser> login(String email, String password) {
    String sql = "select * from users where email = ? and password = ?";
    List<LoginUser> cred = jdbcTemplate.query(sql,
        new BeanPropertyRowMapper<>(LoginUser.class), email, password);
    if (!cred.isEmpty()) {
        return cred;
    } else {
        return null;
    }
}

public List<UsersModel> findAllUsers() {
    String sql = "select * from users";
    return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class));
}

public List<?> searchUser(UsersModel user) {
    String sql = "select * from users where lower(full_name) like (?)";
    String searchCard = "%" + user.getFullName() + "%";
    return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class), searchCard);
}

public UsersModel findUserByEmail(String email) {
    String sql = "select * from users where email = ?";
    List<UsersModel> resp = jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(UsersModel.class));
}
```

User Service Class



```
1 package com.internship2025.changemanagementsystem.Services;
2
3 > import ...
12
13 @Slf4j
14 @Service
15 public class UserService implements Constant {
16
17     private final UsersRepo usersRepo;
18
19     public UserService(UsersRepo usersRepo) { this.usersRepo = usersRepo; }
22
23
24     public ApiResponse<?> saveUsers(UsersModel user) {
25         try {
26             boolean results = usersRepo.save(user);
27             if (results) {
28                 log.info("users save successful");
29                 return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE).build();
30             } else {
31                 log.info("users save failed");
32                 return ApiResponse.builder().message(FAILED_MESSAGE).build();
33             }
34 }
```

```
        return ApiResponse.builder().message(FAILED_MESSAGE).build();
    }

    } catch (Exception e) {
        log.error(e.getMessage(), e);
        return ApiResponse.builder().message(EXCEPTION_MESSAGE).build();
    }
}

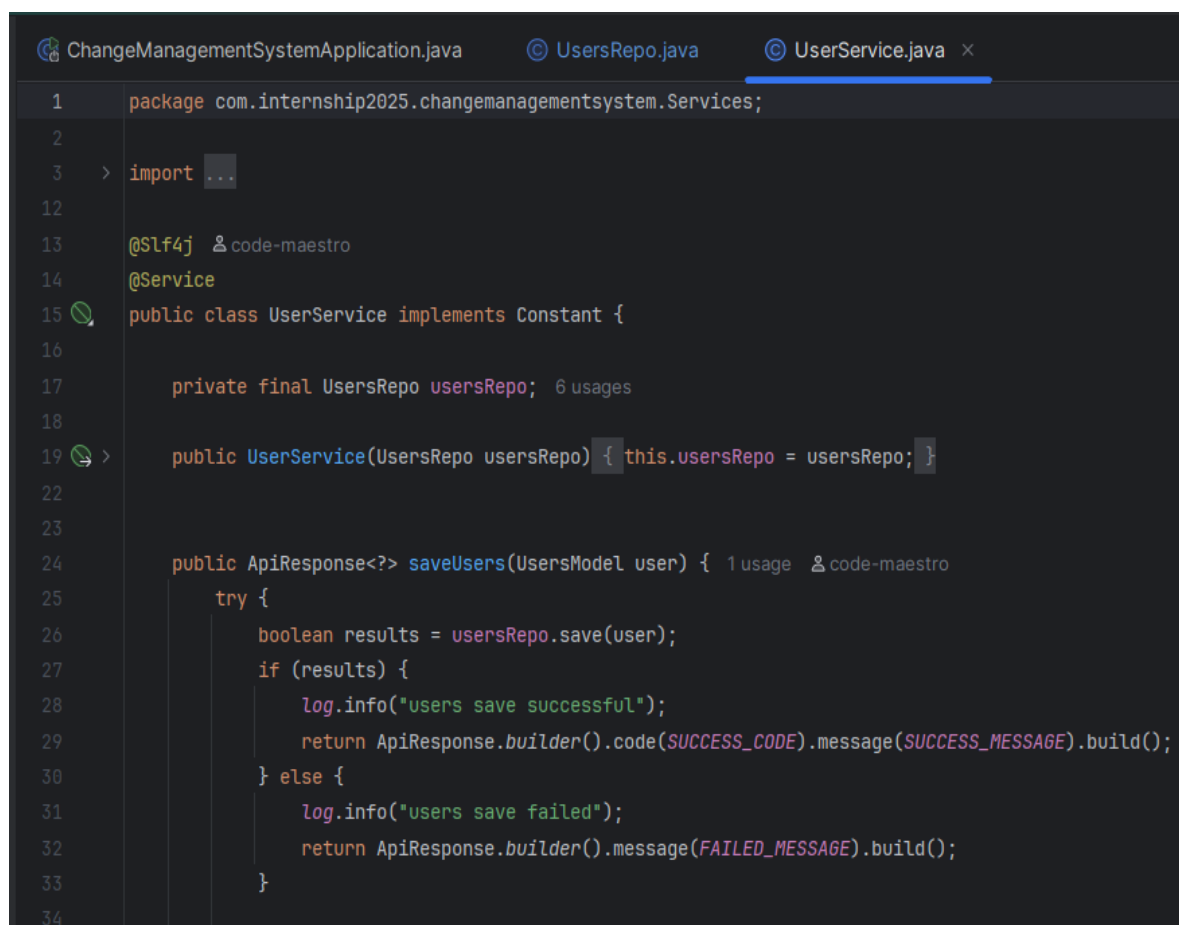
public ApiResponse<?> login(LoginUser loginUser) {
    List<LoginUser> response = usersRepo.login(loginUser.getEmail(), loginUser.getPassword());
    if(response == null){
        return ApiResponse.builder().code(FAILED_CODE)
            .message("Incorrect credentials").build();
    } else
        return ApiResponse.builder().code(SUCCESS_CODE).
            message("Login Successful").build();
}
```

User Service Class Continues

```
public ApiResponse<?> validateLogin(UsersModel user) { 1 usage 2 code-maestro
    try{
        UsersModel responseData = usersRepo.findUserByEmail(user.getEmail());
        if (responseData == null)
            return ApiResponse.builder().code(NO_DATA_FOUND_CODE)
                .message("Invalid Email").build();
        else
            return ApiResponse.builder().code(SUCCESS_CODE)
                .message(SUCCESS_MESSAGE).build();
    }catch(Exception e){
        System.out.println(e.getMessage());
        return ApiResponse.builder().code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE).build();
    }
}

public ApiResponse<?> findUsers(){ 1 usage 2 code-maestro
    try {
        List<UsersModel> usersModelList = usersRepo.findAllUsers();

        if(!usersModelList.isEmpty()){
            System.out.println("users find successful");
        }
    }
}
```



```
ChangeManagementSystemApplication.java  UsersRepo.java  UserService.java x
1  package com.internship2025.changemanagementsystem.Services;
2
3  > import ...
12
13  @Slf4j 2 code-maestro
14  @Service
15  public class UserService implements Constant {
16
17      private final UsersRepo usersRepo; 6 usages
18
19  > public UserService(UsersRepo usersRepo) { this.usersRepo = usersRepo; }
22
23
24  public ApiResponse<?> saveUsers(UsersModel user) { 1 usage 2 code-maestro
25      try {
26          boolean results = usersRepo.save(user);
27          if (results) {
28              log.info("users save successful");
29              return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE).build();
30          } else {
31              log.info("users save failed");
32              return ApiResponse.builder().message(FAILED_MESSAGE).build();
33          }
34      }
}
```

Forget Password Service class

```
© ForgetPasswordService.java × © AuthController.java
1 package com.internship2025.changemanagementsystem.Services;
2
3
4 > import ...
5
6
7
8 @Slf4j  & code-maestro *
9 @Service
10 public class ForgetPasswordService implements Constant {
11
12     private final ForgetPasswordRepo forgetPasswordRepo; 5 usages
13     private final UsersRepo usersRepo; 2 usages
14
15     public ForgetPasswordService(ForgetPasswordRepo forgetPasswordRepo, UsersRepo usersRepo) { & code-maestro
16         this.forgetPasswordRepo = forgetPasswordRepo;
17         this.usersRepo = usersRepo;
18     }
19
20     public ApiResponse<?> validateOtpByEmail(OtpModel otpModel) { 1 usage & code-maestro
21         try {
22             List<OtpModel> otpResponse = forgetPasswordRepo
23                 .selectOtpByEmail(otpModel.getOtpCode(), otpModel.getEmail());
24             if (otpResponse == null || otpResponse.isEmpty()) {
25                 return ApiResponse.builder().code(FAILED_CODE).message("OTP is not valid").build();
26             }
27         }
28     }
29
30 }
```

```
public class ForgetPasswordService implements Constant {
    public ApiResponse<?> validateOtpByEmail(OtpModel otpModel) { 1 usage & code-maestro
    }

    OtpModel foundOtp = otpResponse.get(0);
    if ("Expired".equalsIgnoreCase(foundOtp.getStatus())) {
        return ApiResponse.builder().code(FAILED_CODE).message("OTP is expired").build();
    }
    boolean statusUpdated = forgetPasswordRepo.updateOtpStatus(otpModel);
    if (!statusUpdated) {
        return ApiResponse.builder().code(FAILED_CODE).message("Failed to update OTP status").build();
    } else {
        return ApiResponse.builder().code(SUCCESS_CODE).message(SUCCESS_MESSAGE)
            .build();
    }
} catch (Exception e) {
    log.error("OTP validation failed", e);
    return ApiResponse.builder()
        .code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE + ": " + e.getMessage()).build();
}
}
```


Forget Password Service Continue

```
20 public class ForgetPasswordService implements Constant {
21
22     public ApiResponse<?> forgetPassword(String email) { 1 usage 2 code-maestro *
23         try {
24             if (email == null || email.trim().isEmpty()) {
25                 return ApiResponse.builder().code(FAILED_CODE)
26                     .message("email is incorrect").build();
27             }
28             UsersModel usersModel = usersRepo.findUserByEmail(email.trim());
29             if (usersModel == null) {
30                 return ApiResponse.builder().code(FAILED_CODE)
31                     .message("Email doesn't exist").build();
32             } else {
33                 Integer generatedOTP = OTPNumber();
34                 OtpModel otpModel = new OtpModel();
35                 otpModel.setEmail(email);
36                 otpModel.setOtpCode(generatedOTP);
37                 otpModel.setStatus("Pending");
38                 otpModel.setCreatedDate(LocalDateTime.now());
39
40                 return saveOtpRecord(otpModel);
41             }
42         } catch (Exception e) {
43             return ApiResponse.builder().code(EXCEPTION_CODE)
44                 .message(e.getMessage()).build();
45         }
46     }
47 }
```

```
20 public class ForgetPasswordService implements Constant {
21
22     public ApiResponse<?> forgetPassword(String email) { 1 usage 2 code-maestro *
23         try {
24             if (email == null || email.trim().isEmpty()) {
25                 return ApiResponse.builder().code(FAILED_CODE)
26                     .message("email is incorrect").build();
27             }
28             UsersModel usersModel = usersRepo.findUserByEmail(email.trim());
29             if (usersModel == null) {
30                 return ApiResponse.builder().code(FAILED_CODE)
31                     .message("Email doesn't exist").build();
32             } else {
33                 Integer generatedOTP = OTPNumber();
34                 OtpModel otpModel = new OtpModel();
35                 otpModel.setEmail(email);
36                 otpModel.setOtpCode(generatedOTP);
37                 otpModel.setStatus("Pending");
38                 otpModel.setCreatedDate(LocalDateTime.now());
39
40                 return saveOtpRecord(otpModel);
41             }
42         } catch (Exception e) {
43             return ApiResponse.builder().code(EXCEPTION_CODE)
44                 .message(e.getMessage()).build();
45         }
46     }
47
48     public Integer OTPNumber() { 1 usage 2 code-maestro
49         return 100000 + (int) (Math.random() * 900000); // Generates a number between 100000 and 999999
50     }
51
52     public ApiResponse<?> updatePassword(UsersModel usersModel) { 1 usage 2 code-maestro
53         try {
54             boolean updateResponse = forgetPasswordRepo.updatePassword(usersModel);
55             if (updateResponse) {
56                 return ApiResponse.builder().code(SUCCESS_CODE).message("New password saved").build();
57             } else {
58                 return ApiResponse.builder().code(FAILED_CODE).message(FAILED_MESSAGE).build();
59             }
60         } catch (Exception e) {
61             return ApiResponse.builder().code(EXCEPTION_CODE).message(EXCEPTION_MESSAGE + e.getMessage()).build();
62         }
63     }
64 }
```